Benchmark baseline

Status: initial JIT baseline. This file is a snapshot, not a promise. Rerun the harness with `--benchmark` on the test deployable and replace the table with the new numbers; keep the workloads and the environment line so two runs are comparable.

Harness

`projects/scale-tests` runs the benchmarks:

```
dotnet run --project projects/scale-tests/scale-tests.fsproj -- --benchmark
```

The product base refuses `--benchmark`; the test deployable owns it. The mode prints wall time and `GC.GetAllocatedBytesForCurrentThread` for:

- kernel arithmetic instructions (`LoweredAbstraction`, no word dispatch);
- interpreter arithmetic tokens (`Evaluator`, token boundary);
- a prose book cast (`ProseShell`, parse/lower/load/call);
- text interning and reading (`Store.addText`/`readTextAt`).

Environment

```
Machine: Garuda Linux, 16 logical processors
Runtime: .NET 10.0.12
Date: 2026-11-04
```

Results

Workload	Iterations	Wall time	Allocated
Kernel arithmetic	50,000	650.679 ms	413,240,872 B
Interpreter arithmetic	5,000	122.844 ms	76,320,040 B
Prose book cast	200	1.639 ms	587,192 B
Text intern/read	50,000	313.811 ms	217,210,736 B

This table is one reference run. Two consecutive runs differed by under 8% in wall time; the allocation deltas were identical. A change is judged against this snapshot, not against an intuition.

Initial reading

The baseline is allocation-heavy: kernel arithmetic allocates about 8.1 KiB per four-instruction group, the interpreter about 14.9 KiB per four-token program, and text intern/read about 4.2 KiB per 128-character text. The kernel result also grows the store with every arithmetic result, so part of that allocation is the measured workload, not overhead. Stage 1 should separate store growth from operand-stack churn before claiming a win.

The prose cast is parse/load dominated and is the smallest workload at 200 iterations; keep it as a guard against a regression in the whole path, not as a microbenchmark.